

Algoritmos de Ordenação - QuickSort - MergeSort - HeapSort

Abordagem do **algoritmo Quicksort**, um método de ordenação altamente eficiente desenvolvido por **Hoare** na década de 1960. O conteúdo explica a sua lógica fundamental baseada na estratégia de **dividir para conquistar**, onde o vetor é fragmentado utilizando um **pivô** de referência.

Apresenta-se também a implementação prática da técnica por meio de um **código em linguagem C**, detalhando os ponteiros e a movimentação dos elementos. Por fim, o material estrutura os tópicos essenciais sobre o funcionamento e o particionamento que tornam este processo dinâmico e amplamente utilizado na computação.

Importância da escolha do pivô no desempenho do QuickSort

A escolha do pivô é determinante para o desempenho do Quicksort porque o pivô é a referência utilizada para **particionar o vetor em duas partes independentes**: uma à esquerda (com elementos menores ou iguais ao pivô) e outra à direita (com elementos maiores ou iguais ao pivô).

A importância do pivô no desempenho se reflete nos seguintes aspectos:

- **Balanceamento das Partições:** A eficiência do algoritmo depende de quão equilibrada fica a divisão do vetor em cada etapa. No melhor cenário, a escolha do pivô faz com que **cada partição divida o vetor sistematicamente pela metade**.
- **Altura da Árvore Recursiva:** Quando as partições dividem o vetor ao meio de forma ideal, o processo de divisão ocorre ao longo de $(\lg n)$ níveis.
- **Complexidade de Comparações:** Sabendo que são realizadas aproximadamente (n) comparações em cada um dos $(\lg n)$ níveis de partição, a escolha ideal do pivô leva ao melhor caso de complexidade de comparações, que é $(\Theta(n \lg n))$.
- **Estratégia Prática de Escolha:** Para buscar uma partição equilibrada, uma abordagem comum exemplificada no algoritmo é selecionar o elemento central do intervalo corrente como pivô, definido por $\text{vet}[(\text{esq} + \text{dir})/2]$.

Particionamento na prática do QuickSort

Na prática, o particionamento do Quicksort reorganiza o vetor dividindo-o em duas partes independentes com base em um valor de referência chamado **pivô**. A implementação do algoritmo em estilo *C-like* seleciona como pivô o elemento central do intervalo corrente, obtido por $\text{vet}[(\text{esq} + \text{dir})/2]$.

A execução do particionamento na prática segue os seguintes passos:

- **Inicialização dos ponteiros:** São criados dois ponteiros de índice, em que i é inicializado na extremidade esquerda (esq) e j na extremidade direita (dir) da região a ser ordenada.
- **Avanço do ponteiro esquerdo (i):** O índice i é incrementado ($i++$) sucessivamente enquanto o valor de $\text{vet}[i]$ for estritamente menor que o pivô.
- **Recuo do ponteiro direito (j):** O índice j é decrementado ($j--$) sucessivamente enquanto o valor de $\text{vet}[j]$ for estritamente maior que o pivô.
- **Troca de elementos (swap):** Quando ambos os ponteiros travam em valores fora de ordem e a condição $i \leq j$ é atendida, realiza-se a troca de posição entre $\text{vet}[i]$ e $\text{vet}[j]$.
- **Atualização dos índices:** Imediatamente após a troca, incrementa-se o índice i ($i++$) e decrementa-se o índice j ($j--$).

- **Repetição do laço:** O ciclo de varredura e trocas continua em execução enquanto a condição $i \leq j$ permanecer verdadeira.
- **Cruzamento de ponteiros:** A etapa de particionamento encerra-se no momento em que os ponteiros se cruzam, ou seja, quando i se torna maior do que j ($i > j$).

Como resultado, o particionamento garante que todos os elementos do subvetor de `vet[esq]` até `vet[j]` sejam **menores ou iguais ao pivô**, enquanto todos os elementos de `vet[i]` até `vet[dir]` sejam **maiores ou iguais ao pivô**. A partir desse ponto, o algoritmo efetua chamadas recursivas para `quicksort(esq, j)` e `quicksort(i, dir)` para ordenar as duas partes de forma independente.

Código QuickSort em C

A implementação do algoritmo **Quicksort** em linguagem estilo *C-like* funciona da seguinte forma:

```
void quicksort(int esq, int dir) {
    int i = esq, j = dir, pivo = vet[(esq+dir)/2];
    while (i <= j) {
        while (vet[i] < pivo) i++;
        while (vet[j] > pivo) j--;
        if (i <= j) {
            swap(i, j);
            i++;
            j--;
        }
    }
    if (esq < j)
        quicksort(esq, j);
    if (i < dir)
        quicksort(i, dir);
}
```

Explicação detalhada do código:

- **Assinatura e Chamada Inicial:** A função recebe os parâmetros `esq` e `dir`, que indicam os limites do intervalo atual do vetor a ser ordenado. Na primeira execução do algoritmo para um vetor de tamanho $\backslash(n\backslash)$, a chamada é realizada como `quicksort(0, n-1)`.
- **Inicialização dos Índices e Pivô:** Os ponteiros `i` e `j` são inicializados nas extremidades do intervalo (`esq` e `dir`). O **pivô** é selecionado como o valor presente no elemento central do subvetor, calculado por `vet[(esq+dir)/2]`.
- **Laço de Particionamento (while (i <= j)):**
 - **Varredura da Esquerda:** O laço `while (vet[i] < pivo) i++;` incrementa o índice `i` enquanto encontrar elementos menores que o pivô.

- **Varredura da Direita:** O laço `while (vet[j] > pivô) j--`; decrementa o índice `j` enquanto encontrar elementos maiores que o pivô.
- **Troca (Swap):** Se `i <= j`, significa que foram encontrados elementos fora de ordem em relação ao pivô. A função `swap(i, j)` realiza a troca entre os elementos `vet[i]` e `vet[j]`, seguida do incremento de `i (i++)` e decremento de `j (j--)`.
- **Resultado do Particionamento:** Ao final desse laço, o subvetor fica particionado de modo que todos os elementos de `vet[esq]` até `vet[j]` são **menores ou iguais ao pivô**, e os elementos de `vet[i]` até `vet[dir]` são **maiores ou iguais ao pivô**.
- **Chamadas Recursivas:**
 - Se a condição `esq < j` for verdadeira, a função chama a si mesma recursivamente para ordenar o subvetor à esquerda: `quicksort(esq, j)`.
 - Se a condição `i < dir` for verdadeira, faz-se a chamada recursiva para ordenar o subvetor à direita: `quicksort(i, dir)`.

Outras formas de escolher o pivô

Nos materiais fornecidos, a estratégia apresentada para a escolha do pivô é a seleção do **elemento central do intervalo corrente**, definido pela posição `vet[(esq+dir)/2]`.

*Nota: As estratégias a seguir **não constam nos materiais fornecidos**; portanto, você pode querer verificá-las independentemente.*

Existem outras abordagens amplamente utilizadas na computação para a escolha do pivô:

1. Primeiro ou Último Elemento (`vet[esq]` ou `vet[dir]`):

- **Como funciona:** Escolhe-se sempre o primeiro elemento da sub-região ou o último.
- **Desvantagem:** É uma abordagem vulnerável. Se o vetor já estiver totalmente ordenado ou inversamente ordenado, essa escolha gera partições extremamente desequilibradas (uma partição vazia e outra com $(n-1)$ elementos), levando o algoritmo ao seu pior caso de complexidade, que é $\Theta(n^2)$.

2. Pivô Aleatório (Quicksort Randomizado):

- **Como funciona:** Seleciona-se um índice aleatório dentro do intervalo `[esq, dir]` e utiliza-se o valor daquela posição como pivô.
- **Vantagem:** Dificulta a ocorrência do pior caso, pois o desempenho deixa de depender do arranjo inicial dos dados de entrada. Com isso, obtém-se um tempo esperado de $O(n \lg n)$ na prática.

3. Mediana de Três (*Median-of-Three*):

- **Como funciona:** Examina-se o primeiro elemento (`vet[esq]`), o central (`vet[(esq+dir)/2]`) e o último (`vet[dir]`), escolhendo como pivô o elemento de valor intermediário entre os três.
- **Vantagem:** Evita a escolha de valores extremos (mínimo ou máximo) e melhora significativamente o balanceamento do particionamento, sendo uma das técnicas mais populares para prevenir o pior caso em vetores quase ordenados.

4. Mediana de Medianas (*Tukey's Ninther* / Mediana de Nove):

- **Como funciona:** Para vetores maiores, calcula-se a mediana de pequenas amostras (por exemplo, grupos de 3 elementos) e, em seguida, toma-se a mediana dessas medianas para definir o pivô.
- **Vantagem:** Garante uma estimativa ainda mais precisa da mediana real do vetor, aproximando o particionamento de uma divisão perfeita ao meio.

Complexidade nos piores casos dos 3 algoritmos de ordenação

No contexto dos algoritmos discutidos, a complexidade no pior caso varia de acordo com a estrutura e o funcionamento de cada um:

1. Quicksort

- **Complexidade no Pior Caso:** $\Theta(n^2)$.
- **Causa:** Ocorre quando as escolhas de pivô geram partições **extremamente desequilibradas** a cada etapa (por exemplo, uma partição com (0) elementos e a outra com $(n-1)$ elementos). Isso faz com que a árvore de chamadas recursivas atinja uma profundidade de (n) níveis em vez de $(\lg n)$.
- **Nota:** Como a análise formal do pior caso do Quicksort não consta explicitamente nos trechos das fontes fornecidas (que apresentam o melhor caso com $\Theta(n \lg n)$ comparações), essa informação sobre o pior caso de $\Theta(n^2)$ vem do histórico de conversa/conhecimento externo e você pode querer verificá-la independentemente.

2. Heapsort

- **Complexidade de Comparações no Pior Caso:** $\Theta(n \lg n)$.
- **Causa:** A construção da estrutura exige $(n-1)$ inserções e a ordenação envolve $(n-1)$ remoções da raiz. Como cada inserção ou reconstrução pode percorrer um ramo completo da árvore binária de altura $\Theta(\lg n)$, realizando $\Theta(\lg n)$ comparações, o custo total de comparações é dado por $[(n-1) \cdot \Theta(\lg n)] + [(n-1) \cdot \Theta(\lg n)] = \Theta(n \lg n)$.
- **Complexidade de Movimentações no Pior Caso:** $\Theta(n \lg n)$.

3. Mergesort

A complexidade de tempo no **pior caso do Mergesort** é $\Theta(n \lg n)$ (ou $O(n \log n)$).

- **Divisão Consistente:** Diferentemente do Quicksort, o Mergesort divide o vetor ao meio de forma incondicional em todas as etapas, independentemente de os dados estarem ordenados, desordenados ou em ordem inversa. Isso garante uma árvore de chamada recursiva com profundidade constante de $\lg n$ níveis.
- **Custo de Intercalação (Merge):** Em cada um dos $\lg n$ níveis da árvore, o processo de intercalação dos subvetores realiza no máximo n comparações para unir as partes de forma ordenada.
- **Garantia de Desempenho:** A multiplicação do trabalho por nível (n) pela quantidade de níveis ($\lg n$) resulta na complexidade assintótica de $\Theta(n \lg n)$ no pior caso, assim como no melhor e no caso médio.

Melhor caso do QuickSort

O melhor caso do Quicksort ocorre quando a escolha do pivô resulta em um **particionamento perfeitamente equilibrado** em todas as chamadas do algoritmo.

O funcionamento desse cenário segue a seguinte lógica:

- **Divisão Sistemática pela Metade:** A cada etapa de particionamento, o pivô escolhido divide o vetor corrente **sistematicamente ao meio**, gerando duas subpartições de tamanhos praticamente idênticos.
- **Profundidade Mínima da Recursão:** Devido à divisão ao meio em cada passo, a árvore de chamadas recursivas é mantida equilibrada, atingindo a profundidade mínima possível de $\lg n$ níveis de divisão.
- **Trabalho por Nível:** Em cada nível de recursão, ao somar o trabalho de varredura feito em todas as subpartições daquele passo, são realizadas aproximadamente n comparações.
- **Complexidade Final de Comparações:** Multiplicando o custo de comparações por nível (n) pela quantidade total de níveis ($\lg n$), o total de comparações no melhor caso atinge a complexidade ideal de $\Theta(n \lg n)$.

Existe algum caso em que o quicksort não vai conseguir fazer essas partições e não será possível calcular ou mensurar a complexidade do algoritmo?

Não, não existe nenhum caso em que o Quicksort deixe de realizar o particionamento ou em que seja impossível calcular e mensurar a complexidade do algoritmo.

A lógica do algoritmo e a análise formal da ciência da computação garantem que o particionamento sempre ocorra com sucesso e que o desempenho seja matematicamente mensurável em qualquer cenário:

1. Garantia de Execução do Particionamento

- **Avanço dos Ponteiros:** Durante a etapa de particionamento (`while (i <= j)`), os índices `i` e `j` realizam a varredura a partir das extremidades do vetor.
- **Tratamento de Elementos Iguais:** Os laços internos utilizam comparações estritas (`vet[i] < pivo` e `vet[j] > pivo`). Se um elemento for igual ao pivô (mesmo em vetores com todos os elementos repetidos), a condição do `while` torna-se falsa e a varredura do ponteiro para. A troca é efetuada, `i` é incrementado (`i++`) e `j` é decrementado (`j--`), o que impede que o algoritmo fique preso em laços infinitos.
- **Término e Divisão:** O particionamento encerra-se obrigatoriamente quando os ponteiros se cruzam (`i > j`), dividindo o vetor com sucesso nas sub-regiões de `esq` até `j` e de `i` até `dir`.

2. Mensuração e Cálculo da Complexidade

A complexidade do Quicksort é um conceito matemático baseado no tamanho da entrada (n) e pode ser calculada para qualquer padrão de dados:

- **Melhor Caso:** Ocorre quando cada partição divide o vetor **sistematicamente pela metade**. O número de comparações em cada um dos $\lg n$ níveis de partição resulta na complexidade ideal de $\Theta(n \lg n)$.
- **Pior Caso:** Ocorre quando as escolhas de pivô geram **partições desequilibradas** sucessivas, fazendo com que a profundidade da recursão atinja n níveis, o que leva à complexidade de $\Theta(n^2)$.
- **Caso Médio:** Considera a média de todas as permutações possíveis dos dados de entrada, resultando em uma complexidade esperada de $\Theta(n \lg n)$.

Portanto, **não existem cenários "indeterminados" ou "impossíveis de mensurar"**; qualquer vetor de entrada processado pelo Quicksort obedecerá aos limites teóricos do melhor caso, pior caso ou caso médio do algoritmo.

Se o QuickSort sempre consegue terminar a partição, por que uma escolha ruim do pivô pode levar à complexidade $\Theta(n^2)$? Mostre isso usando uma árvore de chamadas recursivas

O fato de o particionamento do Quicksort sempre terminar com o cruzamento dos ponteiros ($i > j$) garante apenas que a varredura do subvetor atual foi concluída e que os elementos foram reorganizados em relação ao pivô. **O término do particionamento não garante que os subvetores resultantes tenham tamanhos equilibrados.**

Por que uma escolha ruim leva a $\Theta(n^2)$?

No **melhor caso**, a escolha do pivô divide o vetor **sistematicamente pela metade** a cada etapa. Isso gera uma árvore de chamadas recursivas equilibrada com $\lg n$ níveis de profundidade, onde o somatório do trabalho de varredura em cada nível realiza aproximadamente n comparações, resultando em uma complexidade de $\Theta(n \lg n)$.

Quando a escolha do pivô é **ruim** (por exemplo, selecionando sempre o menor ou o maior elemento do subvetor):

1. **Partições Desequilibradas:** O algoritmo divide o vetor em uma subpartição vazia (ou com apenas 1 elemento) e outra subpartição contendo quase todos os elementos restantes ($n-1$).
2. **Profundidade Máxima:** Em vez de reduzir a profundidade da recursão para $\lg n$ níveis, a árvore de chamadas se torna **desequilibrada e linear**, atingindo uma profundidade de n níveis.

Árvore de Chamadas Recursivas no Pior Caso

No pior caso, a árvore de recursão deixa de ser ramificada de forma simétrica e ganha um formato completamente degenerado (semelhante a uma lista encadeada):

Nível 1 (Tamanho n):	quicksort(0, $n-1$)	--> $\sim n$
comparações	/	\

Nível 2 (Tamanho n-1):	quicksort(0, 0)	quicksort(1, n-1)	--> ~ (n-1)
comparações	(caso base)	/	\
Nível 3 (Tamanho n-2):		quicksort(1, 1)	quicksort(2, n-1) -->
~ (n-2) comparações		(caso base)	/
			\
		...	...
			\
Nível n (Tamanho 1):	quicksort(n-1, n-1)	--> ~ 1 comparação	

Cálculo da Complexidade

Somando o número de comparações realizadas no particionamento ao longo dos (n) níveis de profundidade da árvore:

$$\text{Total de Comparações} = n + (n - 1) + (n - 2) + \dots + 2 + 1$$

Pela fórmula da soma de uma progressão aritmética:

$$\text{Total} = \frac{n \cdot (n + 1)}{2} = \frac{n^2 + n}{2} = \mathbf{\Theta(n^2)}$$

(Nota: A demonstração matemática formal através do somatório da série aritmética deriva da análise teórica clássica de complexidade e não consta explicitamente nas lâminas dos materiais fornecidos, devendo ser verificada independentemente).

Portanto, mesmo que o particionamento sempre execute com sucesso, **uma escolha ruim do pivô impede que o problema seja dividido eficientemente**, transformando o custo de divisão e conquista de $\Theta(n \lg n)$ para o pior caso quadrático de $\Theta(n^2)$.

Discuta o que essas fontes dizem sobre a função recursiva com ponteiros esq e dir, no contexto maior do Algoritmo em C-like.

No contexto do **algoritmo em C-like** do Quicksort, a **função recursiva com os parâmetros/índices esq e dir** define a assinatura principal do algoritmo e controla o escopo do vetor a ser ordenado.

A estrutura e o funcionamento dessa função recursiva englobam os seguintes pontos centrais:

1. Assinatura e Chamada Inicial

- A função é declarada com a assinatura `void quicksort(int esq, int dir)`.
- Os parâmetros `esq` e `dir` especificam, respectivamente, o **limite esquerdo (início)** e o **limite direito (fim)** da sub-região do vetor que deve ser processada.
- Na **primeira chamada** para ordenar um vetor de tamanho (n) , a função é invocada passando os limites globais da estrutura: `quicksort(0, n-1)`.

2. Inicialização de Índices e Escolha do Pivô

- Ao entrar na função, os ponteiros/índices de varredura `i` e `j` são inicializados a partir das extremidades recebidas: `int i = esq` e `j = dir`.
- O **pivô** é definido com base em `esq` e `dir`, selecionando o elemento localizado na posição central do intervalo corrente através da expressão `pivo = vet[(esq+dir)/2]`.

3. Divisão do Vetor e Cruzamento de Ponteiros

- Durante o laço de particionamento (`while (i <= j)`), os índices `i` e `j` avançam e recuam, realizando trocas (`swap(i, j)`) até organizarem os elementos em relação ao pivô.
- Ao término do laço de particionamento, os índices se cruzam (`i > j`). Com isso, o intervalo original `[esq, dir]` fica dividido em dois subvetores independentes:
 - **Subvetor à esquerda:** delimitado de `esq` até `j`.
 - **Subvetor à direita:** delimitado de `i` até `dir`.

4. Controle das Chamadas Recursivas (Condições de Parada)

A recursão é disparada por duas verificações condicionais que garantem a existência de subpartições válidas:

- **Subvetor Esquerdo** (`if (esq < j)`): Se o limite `esq` for estritamente menor que `j`, realiza-se a chamada recursiva `quicksort(esq, j)` para ordenar a metade esquerda.
- **Subvetor Direito** (`if (i < dir)`): Se o limite `dir` for estritamente maior que `i`, realiza-se a chamada recursiva `quicksort(i, dir)` para ordenar a metade direita.
- Quando uma condição resulta em `false` (isto é, `esq >= j` ou `i >= dir`), o subvetor correspondente atingiu o caso base (tamanho 1 ou vazio) e o ramo recursivo se encerra.

5. Execução Prática no Algoritmo C-like

- Na demonstração visual do algoritmo para um vetor de 16 elementos (índices 0 a 15) com pivô 10, o particionamento inicial resulta em `j = 7` e `i = 8`.
- Como a condição `esq < j` (`0 < 7`) é verdadeira, o algoritmo faz a chamada recursiva `quicksort(0, 7)`.
- Nessa nova execução recursiva, `esq` e `dir` assumem os valores 0 e 7, e um novo pivô é recalculado para esse subintervalo: `vet[(0+7)/2]`, que resulta no pivô de índice 3.
- Após o encerramento do processamento do lado esquerdo, a execução retorna à chamada original e testa `i < dir` (`8 < 15`), invocando `quicksort(8, 15)` para ordenar a partição direita.

Como funciona o processo de memória pro quick, merge e heap sort?

O funcionamento do processo de memória para os algoritmos de ordenação **Quicksort**, **Mergesort** e **Heapsort** diferencia-se principalmente pelo uso de **alocação de vetores auxiliares** e pelo consumo de **pilha de execução recursiva**:

1. Quicksort

- **Ordenação *In-place* (no próprio vetor):** O Quicksort reorganiza os elementos diretamente dentro do vetor original (`vet`) por meio de trocas de posição (*swaps*) entre os ponteiros `i` e `j` durante o particionamento. Ele não exige a criação de um vetor secundário para manipular os dados.
- **Consumo na Pilha de Recursão:** O uso de memória do Quicksort decorre da **pilha de chamadas recursivas** da função `quicksort(esq, dir)`.
 - No **melhor caso e caso médio**, a divisão do vetor gera uma árvore de recursão com $\lg n$ níveis de profundidade, resultando em um consumo de memória de pilha de $O(\lg n)$.
 - (*Informação externa às fontes fornecidas*): No pior caso (quando as partições ficam desequilibradas), a profundidade da recursão pode chegar a $O(n)$.

2. Mergesort

(Nota: As fontes fornecidas para o Mergesort contêm apenas a capa da apresentação. As informações abaixo baseiam-se no conhecimento teórico clássico sobre o algoritmo e você pode querer verificá-las independentemente).

- **Uso de Vetor Auxiliar:** Ao contrário do Quicksort e do Heapsort, o Mergesort padrão **não é um algoritmo *in-place***. Durante a fase de intercalação (*merge*), é necessário alocar temporariamente um **vetor auxiliar de tamanho $O(n)$** para fundir os subvetores de forma ordenada antes de copiar os dados de volta para o vetor original.
- **Pilha de Recursão:** Por se tratar de um algoritmo de divisão e conquista que divide o problema sempre ao meio, ele consome adicionalmente $O(\lg n)$ de espaço na pilha de chamadas recursivas.

3. Heapsort

- **Ordenação *In-place* (In-situ):** O Heapsort trabalha **exclusivamente no vetor original**, interpretando os índices do próprio vetor como uma árvore binária (Heap Máximo) e realizando trocas diretas.
- **Execução Iterativa:** A construção (*construir*) e a manutenção do heap (*reconstruir*) são feitas por meio de laços iterativos (`for` e `while`). Por não utilizar recursão, **não há consumo de memória na pilha de chamadas**.
- **Complexidade de Espaço Auxiliar:** O espaço de memória adicional exigido para a ordenação é $O(1)$ (constante).
- (*Exceção pontual nas fontes*): Alocação dinâmica de memória via `malloc` só é apresentada em rotinas auxiliares específicas fora da ordenação principal, como na união de dois heaps distintos para formar um terceiro.

Resumo Comparativo de Memória

Algoritmo	Requer Vetor Auxiliar?	Estrutura de Execução	Memória Auxiliar Necessária
Quicksort	Não (<i>In-place</i>)	Recursivo	$O(\lg n)$ na pilha de recursão

Mergesort	Sim (para intercalação)*	Recursoivo	$\backslash(O(n))$ (vetor auxiliar + pilha)*
Heapsort	Não (<i>In-place</i>)	Iterativo (<code>for/while</code>)	$\backslash(O(1))$ (memória constante)

*Detalhes sobre o vetor auxiliar do Mergesort não constam nos trechos das fontes fornecidas.

Como o Heapsort gerencia a memória?

O **Heapsort** gerencia a memória organizando e manipulando os elementos **diretamente no próprio vetor de entrada**, caracterizando uma ordenação *in-place*.

A gestão de memória do algoritmo ocorre segundo os seguintes princípios:

- **Mapeamento Aritmético da Árvore (Sem Ponteiros Especiais):** A estrutura de árvore binária do **Heap Máximo** é mapeada sobre os próprios índices do vetor, eliminando a necessidade de alocar nós dinâmicos com ponteiros explícitos de memória.
- **Indexação com Base em 1 (1-based index):** O elemento `vet` armazena a raiz do heap, o pai de qualquer nó no índice `i` é localizado em `vet[i/2]`, e seus filhos ficam posicionados em `vet[2i]` e `vet[2i+1]`.
- **Indexação com Base em 0 (0-based index):** O elemento `vet` atua como a raiz, o pai do nó `i` situa-se na posição `vet[(i-1)/2]`, e os filhos ocupam os índices `vet[2i+1]` e `vet[2i+2]`.
- **Execução Iterativa (Sem Consumo de Pilha Recursiva):** As rotinas principais de construção (`construir`) e de manutenção (`reconstruir`) utilizam laços `for` e `while`, de modo que o algoritmo não realiza chamadas recursivas e não consome memória na pilha de execução.
- **Alocação Dinâmica Apenas em Operações Auxiliares:** A alocação dinâmica de memória suplementar por meio de `malloc` é restrita a funções auxiliares específicas fora do processo normal de ordenação, como no procedimento de união de dois heaps para formar um terceiro.

Quais as principais diferenças entre os três algoritmos?

As principais diferenças entre o Quicksort, o Mergesort e o Heapsort concentram-se no paradigma de funcionamento, na gestão de memória e nas garantias de complexidade de tempo:

1. Estratégia e Paradigma Principal

- **Quicksort:** É um algoritmo do tipo dividir para conquistar. Ele funciona selecionando um valor de referência chamado pivô (como a posição central `vet[(esq+dir)/2]`) e realizando o particionamento dos elementos. Em seguida, processa os subvetores resultantes por meio de chamadas recursivas.
- **Mergesort:** (*Nota: As informações sobre a execução do Mergesort baseiam-se no histórico da conversa e em conhecimento externo, pois a fonte fornecida contém apenas o título do tópico*). É também um algoritmo de dividir para conquistar, mas que divide o

vetor ao meio de forma incondicional e realiza a ordenação propriamente dita durante a fase de intercalação (*merge*).

- **Heapsort:** É um algoritmo de ordenação por seleção. Ele organiza o vetor na forma de uma árvore binária chamada Heap Máximo, onde o maior elemento é mantido na raiz. O algoritmo troca o elemento da raiz com a última posição não ordenada do vetor e executa uma rotina de reconstrução do heap para repetir o processo.

2. Gerenciamento e Uso de Memória

- **Quicksort:** Realiza a reorganização dos elementos diretamente no próprio vetor (*in-place*), porém consome memória na pilha de chamadas recursivas da função `quicksort(esq, dir)`.
- **Mergesort:** (*Informação externa ao material impresso*) Não é um algoritmo *in-place*, exigindo a alocação de um vetor auxiliar de tamanho $\mathcal{O}(n)$ para fundir os subvetores, além do espaço ocupado pela pilha de recursão.
- **Heapsort:** Trabalha de forma estritamente ***in-place*** e iterativa, utilizando laços `for` e `while` nas funções `construir` e `reconstruir`. Por não ser recursivo, não consome memória na pilha de execução e exige espaço auxiliar constante $\mathcal{O}(1)$.

3. Complexidade e Garantias de Desempenho

- **Quicksort:**
 - **Melhor Caso:** Atinge $\mathcal{O}(n \lg n)$ comparações quando as partições dividem o vetor sistematicamente pela metade ao longo de $\lg n$ níveis.
 - **Pior Caso:** Degenera para $\mathcal{O}(n^2)$ quando escolhas ruins de pivô geram partições extremamente desequilibradas (*conforme analisado no histórico da conversa*).
- **Mergesort:** (*Informação externa*) Garante desempenho de $\mathcal{O}(n \lg n)$ em todos os casos (pior, médio e melhor), pois a divisão ao meio independe do arranjo inicial dos dados.
- **Heapsort:** Oferece garantia teórica de $\mathcal{O}(n \lg n)$ tanto no pior caso quanto no melhor caso de comparações e movimentações. Isso ocorre porque a estrutura de árvore possui altura delimitada por $\mathcal{O}(\lg n)$ e a ordenação realiza $(n-1)$ inserções e $(n-1)$ remoções/reconstruções.

Resumo Comparativo

Algoritmo	Paradigma	Estrutura de Execução	Memória Auxiliar	Pior Caso	Melhor Caso
Quicksort	Dividir para Conquistar	Recursivo	$\mathcal{O}(\lg n)$ (pilha)*	$\mathcal{O}(n^2)$ *	$\mathcal{O}(n \lg n)$
Mergesort	Dividir para Conquistar *	Recursivo *	$\mathcal{O}(n)$ (vetor aux.)*	$\mathcal{O}(n \lg n)$ *	$\mathcal{O}(n \lg n)$ *
Heapsort	Ordenação por	Iterativo	$\mathcal{O}(1)$	$\mathcal{O}(n \lg n)$	$\mathcal{O}(n \lg n)$

Seleção (for/while) (constante) n) n)

**Detalhes marcados com asterisco advêm do histórico de conversa/conhecimento externo e você pode querer verificá-los independentemente.*

Como comparar o Quicksort com outros algoritmos?

A comparação entre o Quicksort e outros algoritmos de ordenação (como o Heapsort e o Mergesort) pode ser estruturada com base no paradigma de projeto, na complexidade de tempo e no gerenciamento de memória:

1. Paradigma e Mecanismo de Funcionamento

- **Quicksort:** É um algoritmo do tipo dividir para conquistar proposto por Hoare em 1960. Ele seleciona um elemento como pivô e particiona o vetor reorganizando os elementos de modo que os menores ou iguais ao pivô fiquem à esquerda e os maiores ou iguais fiquem à direita, processando as subpartições recursivamente.
- **Heapsort:** É um algoritmo de ordenação por seleção que utiliza a estrutura de dados Heap Máximo para identificar e selecionar o maior elemento de forma eficiente. Ele coloca o maior valor na raiz do heap, troca-o com a última posição não ordenada do vetor e executa uma rotina iterativa de reconstrução para restabelecer a propriedade do heap.
- **Mergesort** (*informação externa às fontes fornecidas; você pode querer verificá-la independentemente*): Também segue o paradigma de dividir para conquistar, porém divide o vetor incondicionalmente ao meio e realiza a ordenação propriamente dita durante a fase de intercalação (*merge*).

2. Complexidade de Tempo e Garantias de Desempenho

- **Melhor Caso:**
 - **Quicksort:** Atinge a complexidade ideal de $\Theta(n \lg n)$ comparações quando cada partição divide o vetor sistematicamente pela metade ao longo de $\lg n$ níveis de recursão.
 - **Heapsort:** Apresenta complexidade de $\Theta(n \lg n)$ comparações e movimentações.
 - **Mergesort** (*informação externa*): Atinge $\Theta(n \lg n)$.
- **Pior Caso:**
 - **Quicksort:** Degenera para $\Theta(n^2)$ (*informação externa às fontes/histórico*) caso as escolhas de pivô gerem partições sucessivamente desequilibradas.
 - **Heapsort:** Oferece garantia teórica de $\Theta(n \lg n)$ tanto no número de comparações quanto no número de movimentações/trocas. Isso ocorre porque a altura da árvore binária é delimitada por $\Theta(\lg n)$ e a ordenação executa $\Theta(n-1)$ inserções e $\Theta(n-1)$ reconstruções.
 - **Mergesort** (*informação externa*): Garante desempenho de $\Theta(n \lg n)$ em qualquer padrão de entrada.

3. Uso de Memória e Estrutura de Execução

- **Quicksort:** Reorganiza os elementos no próprio vetor (*in-place*), porém depende do consumo de pilha de chamadas recursivas para gerenciar a ordenação dos subvetores.
- **Heapsort:** Trabalha de forma estritamente *in-place* e iterativa, empregando laços *for* e *while* nas rotinas de construção e reconstrução do heap. Por não utilizar recursão, exige espaço auxiliar constante de $O(1)$ sem utilizar a pilha de execução.
- **Mergesort (*informação externa*):** Não é um algoritmo *in-place*, exigindo a alocação de um vetor auxiliar de tamanho $O(n)$ para fundir as partes ordenadas.

Tabela Comparativa Resumida

Critério	Quicksort	Heapsort	Mergesort *
Tipo / Paradigma	Dividir para Conquistar	Ordenação por Seleção	Dividir para Conquistar *
Melhor Caso	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$ *
Pior Caso	$O(n^2)$ *	$O(n \lg n)$	$O(n \lg n)$ *
Estrutura	Recursivo	Iterativo (<i>for/while</i>)	Recursivo *
Memória Auxiliar	Pilha de recursão ($O(\lg n)$)	$O(1)$ (constante, <i>in-place</i>)	Vetor Auxiliar ($O(n)$) *

*Os pontos marcados com asterisco advêm de conhecimento externo/histórico de conversa e não constam nas páginas das fontes fornecidas.

Pseudocódigo, explicação breve da função *merge* (intercalação).

Como exatamente dois vetores ordenados são unidos?

1. MergeSort — A Função de Intercalação (*merge*)

Como dois subvetores ordenados são unidos?

A intercalação é o processo de pegar dois subvetores adjacentes já ordenados (vamos chamá-los de *Esquerda* e *Direita*) e fundi-los em um único bloco ordenado.

1. **Ponteiros de Controle:** Define-se um ponteiro *i* para o início da metade esquerda, um ponteiro *j* para o início da metade direita e um ponteiro *k* para a posição atual no vetor final (ou auxiliar).
2. **Comparação Direta:** Em cada passo, compara-se o elemento apontado por *i* com o elemento apontado por *j*.
3. **Cópia do Menor:** O menor elemento entre os dois é copiado para a posição *k*. O ponteiro do subvetor de onde o elemento foi retirado avança uma posição, assim como o ponteiro *k*.

4. **Cópia dos Restantes:** Quando os elementos de um dos subvetores se esgotarem, os elementos restantes do outro subvetor são copiados em sequência para o vetor final.

Pseudocódigo em C-like da Intercalação:

```
void merge(int vet[], int esq, int meio, int dir) {  
    int n1 = meio - esq + 1;  
    int n2 = dir - meio;  
  
    // Vetores temporários para armazenar as duas metades  
    int E[n1], D[n2];  
  
    for (int i = 0; i < n1; i++) E[i] = vet[esq + i];  
    for (int j = 0; j < n2; j++) D[j] = vet[meio + 1 + j];  
  
    int i = 0, j = 0, k = esq;  
  
    // Compara e intercala os elementos de E e D de volta em vet  
    while (i < n1 && j < n2) {  
        if (E[i] <= D[j]) {  
            vet[k] = E[i];  
            i++;  
        } else {  
            vet[k] = D[j];  
            j++;  
        }  
        k++;  
    }  
}
```

```

// Copia eventuais elementos restantes da metade esquerda
while (i < n1) {
    vet[k] = E[i];
    i++;
    k++;
}

// Copia eventuais elementos restantes da metade direita
while (j < n2) {
    vet[k] = D[j];
    j++;
    k++;
}
}

```

2. HeapSort — A Função de Reconstrução / Heapify (reconstruir)

No Heapsort, após mover a raiz (maior elemento) para o final do vetor, um novo elemento (geralmente menor) passa a ocupar a raiz provisoriamente¹. A função reconstruir (também conhecida teoricamente como heapify) faz esse elemento "descer" na estrutura de árvore até restabelecer a propriedade de Heap Máximo.

A) Código do Algoritmo em C-like

A função reconstruir é implementada de forma iterativa utilizando laços while:

```

void reconstruir(int tam) {
    int i = 1; // Começa sempre na raiz
    while (hasFilho(i, tam) == true) {
        int filho = getMaiorFilho(i, tam);
        if (vet[i] < vet[filho]) {
            swap(i, filho); // Troca o pai com seu maior filho
            i = filho;      // O elemento "desce" para o índice do filho
        } else {

```

```

        i = tam;    // Encerra a reconstrução quando o pai é >= filhos
    }
}
}

```

(Nota: O código utiliza indexação baseada em 1, onde a raiz fica em `vet4`, o filho esquerdo de *i* fica em `vet[2i]` e o direito em `vet[2i+1]`).

B) Passo a Passo Visual da "Descida" do Elemento

Imagine um heap de tamanho reduzido `tam = 5` onde a raiz acabou de receber um valor pequeno :

Árvore inicial (violando a regra do Heap Máximo):

```

    (20) <-- i = 1 (raiz)
      /  \
    (101) (30)
     /  \
    (63) (47)

```

Vetor: 2 (índices 1 a 5)

1. Início na Raiz (i = 1):

- O ponteiro *i* começa em 1 (valor 20).
- A função `hasFilho(1, 5)` retorna true.
- `getMaiorFilho(1, 5)` compara os filhos (`vet9 = 101` e `vet5 = 30`) e identifica que o maior filho é o 101 (índice 2).

2. Primeira Troca e Descida:

- Avalia-se `if (vet[i] < vet[filho]) 20 < 101` (Verdadeiro).
- Executa-se `swap(1, 2)`: o valor 20 troca de lugar com o 101.
- O ponteiro é atualizado para *i* = 2: o valor 20 desceu um nível na árvore.

Árvore após a 1ª descida:

```

    (101) <-- Nova raiz em vet[4]
      /  \
    (20)  (30) <-- i = 2
     /  \
    (63) (47)

```

Vetor: 2

1. Segunda Comparação no Nível Inferior (i = 2):

- Em *i* = 2, `hasFilho(2, 5)` ainda é true (filhos nos índices 4 e 5).

- `getMaiorFilho(2, 5)` compara os dois filhos (`vet13 = 63` e `vet14 = 47`) e aponta o maior filho como 63 (índice 4).
- Avalia-se `if (vet[i] < vet[filho]) 20 < 63` (Verdadeiro).
- Executa-se `swap(2, 4)`: o valor 20 troca com o 63.
- O ponteiro é atualizado para `i = 4`: o valor 20 desce mais um nível.

Árvore após a 2ª descida:

```

      (101)
     /   \
  (63)  (30)
   /   \
(20) (47) <-- i = 4

```

Vetor: 2

1. Término do Processo:

- Em `i = 4`, testa-se `hasFilho(4, 5)`.
- Como a posição de um filho seria , ultrapassando o tamanho atual do heap (`tam = 5`), a função retorna `false`.
- O laço termina, pois o elemento chegou a uma folha e a propriedade de Heap Máximo foi totalmente restaurada .

C) Princípios-Chave da Reconstrução

- **Troca sempre pelo MAIOR filho:** Se trocássemos pelo menor filho, o maior filho continuaria sendo superior ao pai, mantendo o heap inválido.
- **Parada Automática:** O elemento para de descer assim que se tornar maior ou igual aos seus filhos ou quando alcançar o nível das folhas.